ARKA-Lite

# Proyecto final · Upskilling Backend Java

Equipo arka-litos: dos microservicios Spring Boot, sus bases de datos y su plataforma, contados de punta a punta.

SIN DEMOSTRACIÓN · CONTENIDO Y GUION

PARTES


**1. Fundamentos** Lo que nos quedó: analizar antes de programar y buenas prácticas para cualquier lenguaje.

**2. Arquitectura** Lo que nos quedó: cada servicio con sus datos, y mensajes entre ellos.

**3. Base de datos** Dos bases PostgreSQL, el ORM, los datos de conexión y las transacciones.

**4. Interconexión de servicios** Cómo se hablan los servicios: síncrono, asíncrono, Outbox e idempotencia.

**5. DevOps: Docker y Kubernetes** Docker empaqueta, el pipeline prueba cada cambio y Kubernetes administra.

**6. Observabilidad** Cómo sabemos qué pasa adentro: salud, registros y un número de seguimiento.


Generado desde docs/presentation-data/contenido. Cada lámina tiene: lo que se ve y el guion: qué decir, con tus palabras, si preguntan y para pasar.

# Apertura

## 1. Del endpoint al clúster

*Presenta: Luis Cardoze*

**QUÉ DECIR**

Buenos días. Ya vimos los fundamentos y una primera mirada a los temas que ahora vamos a desarrollar: cómo se guardan y se protegen los datos, cómo se comunican los servicios y cómo llevamos una aplicación del código a la operación.

Todo lo vamos a ver con ARKA-Lite, nuestro proyecto final del curso.

Veamos cómo organizamos el recorrido.

**SI SE TE OLVIDA, DI ESTO**

- Retomamos lo que ya vieron para profundizar en los datos, la comunicación entre servicios y la operación de la aplicación.
- Lo vemos con ARKA-Lite, nuestro proyecto final.

**QUÉ HACER (NO SE DICE)**

- El moderador ya presentó al equipo: saludar y entrar directo al tema, sin volver a presentarse.
- Antes de empezar, abrir este PPTX en la computadora de la sala y quitar la última lámina, que es solo para el equipo.

**SI HAY TIEMPO**

- ARKA-Lite nació como laboratorio del curso, así que aplicamos cada tema al mismo sistema. Primero hicimos una operación y después agregamos la base, la comunicación, los contenedores, el pipeline y los registros.
- Trabajamos como un equipo real. Cada cambio se hizo por separado, otra persona lo revisó y las pruebas automáticas lo validaron antes de incorporarlo.

## 2. Qué vamos a ver

*Presenta: Luis Cardoze*

**QUÉ DECIR**

Primero les contamos lo que nos quedó de los fundamentos y la arquitectura, porque esos temas ya los vieron en la presentación anterior. Dedicaremos más tiempo a otros tres temas: la base de datos, cómo se comunican los servicios sin perder información y DevOps, que abarca cómo construimos, publicamos y operamos el sistema.

Después vemos la observabilidad, que es cómo sabemos lo que pasa adentro, y cerramos con los resultados.

Empecemos con lo que nos quedó de los fundamentos.

**SI SE TE OLVIDA, DI ESTO**

- Primero compartimos lo que nos quedó de los fundamentos y la arquitectura.
- Nos detenemos en la base de datos, la comunicación entre servicios y DevOps.

**SI HAY TIEMPO**

- Las partes generales son breves porque dedicaremos más tiempo a los datos, la comunicación entre sistemas y la operación, que son los temas con mayor valor práctico.

# 1 · Fundamentos

Lo que nos quedó: analizar antes de programar y buenas prácticas para cualquier lenguaje.

## 3. Lo primero: analizar antes de programar

*Presenta: Luis Cardoze*

**QUÉ DECIR**

De los fundamentos, lo primero que nos quedó es analizar antes de programar. Antes de escribir código nos hicimos tres preguntas sobre el negocio.

Primero, qué tiene que pasar. Una persona registra una solicitud, como una incidencia o un pedido de cambio, la envía, y otra persona la aprueba o la rechaza. De ahí salieron cuatro estados: borrador, enviada, aprobada y rechazada.

Segundo, quién hace cada paso. Quien solicita envía, el sistema avisa y otra persona decide. Gestionar la solicitud y producir el aviso son trabajos distintos, y por eso los separamos en dos servicios.

Tercero, qué no se puede permitir. Estas son las reglas de ARKA-Lite: una solicitud solo puede enviarse cuando está en borrador y una sola vez. Solo una solicitud enviada puede aprobarse o rechazarse. Cada regla quedó con su prueba automática.

Con el qué ya definido, pasamos al cómo: escribir código que se pueda cambiar con confianza.

**SI SE TE OLVIDA, DI ESTO**

- Antes de programar, aclaramos el proceso y sus responsables.
- Las respuestas produjeron cuatro estados y dos servicios.
- Cada regla importante quedó cubierta por una prueba.

**SI PREGUNTAN**

**¿Por qué los avisos van en otro servicio?**

Porque cambian por motivos distintos. Así, el trámite puede seguir funcionando aunque el servicio de avisos esté caído.

**SI HAY TIEMPO**

- La pregunta sobre lo que no podíamos permitir nos permitió definir las reglas antes de escribir código. De otra manera, habríamos descubierto esos casos más tarde y corregirlos habría costado más.
- La base de datos también protege los cuatro estados, porque la columna correspondiente solo acepta esos valores.

- Cuando arranca, el sistema registra dos solicitudes de ejemplo. El pipeline utiliza una para confirmar que el servicio realmente inició.

## 4. Código mantenible: buenas prácticas

*Presenta: Luis Cardoze*

**QUÉ DECIR**

Otra cosa que nos quedó es cómo escribir código fácil de mantener. Son seis prácticas que sirven en cualquier lenguaje, ya sea Java, C# o PL/SQL.

La primera es que cada pieza hace una sola tarea: la que recibe el pedido no decide, y la que decide no guarda. La segunda es conectar las piezas desde afuera, para poder cambiarlas o probarlas por separado. La tercera es dejar las reglas del negocio en un solo lugar, sin repetirlas en la pantalla y en la base.

La cuarta es que todos los errores sigan el mismo formato y expliquen qué pasó. La quinta es usar los nombres del negocio, para que el código se lea como el proceso. Y la sexta es que cada regla tenga su prueba automática.

Es como una cocina ordenada, donde cada persona tiene su puesto y cada cosa su lugar. Así, cuando cambia una regla, tocamos un solo lugar y sabemos enseguida si algo dejó de funcionar.

Esto es lo que nos llevamos a cualquier proyecto.

**SI SE TE OLVIDA, DI ESTO**

- Estas prácticas sirven en cualquier lenguaje.
- Cada pieza hace una tarea y cada regla vive en un lugar.
- Las pruebas nos avisan cuando un cambio rompe algo.

**SI PREGUNTAN**

**¿Esto no hace el código más largo?**

Puede producir más piezas, pero cada una es corta y se entiende por separado. Esa inversión se recupera desde el primer cambio.

**SI HAY TIEMPO**

- Por ejemplo, la pieza que recibe los pedidos no toma decisiones, sino que los pasa a las reglas. Si cambia la regla de envío, modificamos un lugar y ejecutamos su prueba.
- Las mismas ideas sirven con procedimientos almacenados. Podemos tener un procedimiento por tarea, con nombres del negocio y mensajes de error claros.

## 5. Fundamentos: lo que nos quedó

*Presenta: Luis Cardoze*

**QUÉ DECIR**

Nos quedamos con tres ideas. Primero se analiza y después se programa, porque tener claras las reglas desde el inicio evita rehacer trabajo.

Segundo, separamos responsabilidades para que un cambio de regla toque un solo lugar y una prueba nos avise si algo se rompió. Tercero, las reglas del negocio no deben depender de la herramienta. Así siguen siendo las mismas si cambia la base de datos o la tecnología de la aplicación.

Estas tres prácticas se pueden aplicar en los sistemas actuales, sin importar el lenguaje ni cómo esté organizada la aplicación.

Ahora le paso la palabra a Ricardo, que nos muestra la arquitectura del sistema.

**SI SE TE OLVIDA, DI ESTO**

- Primero analizamos y después programamos.
- Separar responsabilidades facilita los cambios.
- Las reglas deben sobrevivir a un cambio de herramienta.

**SI HAY TIEMPO**

- Si tuviéramos que comenzar con una práctica en un sistema existente, pondríamos las reglas del negocio en un solo lugar y agregaríamos sus pruebas. Es una inversión que devuelve valor con rapidez.

# 2 · Arquitectura

Lo que nos quedó: cada servicio con sus datos, y mensajes entre ellos.

## 6. El sistema de un vistazo

*Presenta: Ricardo Fábrega*

**QUÉ DECIR**

Gracias, Luis. ARKA-Lite no es una aplicación grande, sino varias piezas pequeñas, cada una con su responsabilidad. Las vemos por capas, de arriba hacia abajo.

Arriba está la pantalla que usa la persona. No guarda información, solo les pasa los pedidos a los servicios.

En el centro están los dos microservicios, es decir, dos aplicaciones pequeñas con una función cada una. Uno maneja las solicitudes y sus estados, y el otro produce los avisos.

Debajo, cada servicio tiene su propia base de datos, y ninguno entra en la base del otro. Al final está la plataforma, donde todo corre, se prueba y se revisa.

Es como un edificio donde cada piso tiene su función, y por eso se puede cambiar uno sin reconstruir los demás.

Con el mapa claro, veamos cómo colaboran esas piezas en la práctica.

**SI SE TE OLVIDA, DI ESTO**

- El sistema tiene cuatro capas.
- Cada servicio tiene una tarea y una base propia.
- Las capas se pueden cambiar de forma independiente.

**SI PREGUNTAN**

**¿Por qué no comparten una sola base de datos?**
Porque cada servicio debe controlar sus propios datos. Así puede cambiar sin romper al otro.

**SI HAY TIEMPO**

- La pantalla es delgada a propósito, porque no guarda datos ni toma decisiones. Si la reemplazamos en el futuro, los servicios no tienen que cambiar.
- Los dos servicios comparten la misma plataforma, el mismo pipeline y el mismo lugar para revisar los registros. Eso reduce el costo operativo de mantenerlos separados.

---

## 7. Cómo funciona en la práctica

*Presenta: Ricardo Fábrega*

**QUÉ DECIR**

Sigamos los números. La persona usa la pantalla, y la pantalla le pide al servicio de solicitudes que cree o envíe una solicitud. El servicio la guarda en su propia base.

Cuando la solicitud se envía, el servicio produce un mensaje pequeño para avisarle a notificaciones. A ese mensaje lo llamamos evento. Notificaciones crea el aviso y anota que ya recibió ese evento, y la pantalla consulta los avisos para mostrarlos.

Ninguna flecha une las dos bases. Es como dos departamentos que se intercambian notas, mientras cada uno mantiene el control de sus propios archivos.

Ya vimos cómo se conectan por fuera. Ahora veamos cómo está organizado cada servicio por dentro.

**SI SE TE OLVIDA, DI ESTO**

- La pantalla pide el trabajo y el servicio guarda los datos.
- Entre los servicios viaja un mensaje pequeño llamado evento.
- Los servicios se comunican sin compartir sus bases.

**SI PREGUNTAN**

**¿Qué pasa si notificaciones está caído en el paso cuatro?**
Solicitudes sigue funcionando y el evento queda guardado hasta que pueda entregarse. Lo vemos en la parte de interconexión.

**SI HAY TIEMPO**

- El mensaje entre los servicios lleva tres datos, que son el número del evento, el número de la solicitud y su tipo. Al mantenerlo pequeño, resulta más sencillo coordinar ambos lados.
- El formato del mensaje existe en los dos servicios. Si cambia, debemos actualizar ambos al mismo tiempo, porque es lo único que comparten.

## 8. Por dentro de cada servicio: hexagonal

*Presenta: Ricardo Fábrega*

**QUÉ DECIR**

Cada servicio usa arquitectura hexagonal. El equipo anterior ya la explicó, así que les contamos solo lo que nos quedó.

Las reglas del negocio van en el centro, y todo lo demás se conecta alrededor, como un enchufe. Por un lado entran los pedidos de la pantalla, y por el otro se conectan la base de datos y el servicio de notificaciones. Así, las reglas no dependen de los detalles de esas conexiones.

Donde más nos sirvió fue en las pruebas. Cambiamos la conexión con la base por una versión en memoria, y con eso probamos todas las reglas en segundos, sin levantar una base.

Y esto solo ordena el código por dentro. Desde afuera, el servicio se instala y se usa como cualquier otro.

Esto es lo que nos quedó de la arquitectura.

**SI SE TE OLVIDA, DI ESTO**

- Las reglas quedan en el centro.
- Las conexiones externas se pueden reemplazar.
- Así probamos las reglas rápidamente y sin una base.

**SI PREGUNTAN**

**¿Esto se parece a llamar un procedimiento almacenado?**
Se parece en que quien llama conoce el nombre y los parámetros, pero no los detalles internos. Aquí el negocio define el contrato y la base se adapta a él.

**SI HAY TIEMPO**

- Dejamos versiones alternativas de algunos conectores, como una forma de avisar sin reintentos. Eso demuestra que podemos reemplazarlos sin modificar las reglas.
- La idea se parece a varias aplicaciones que llaman un procedimiento por su nombre sin conocer cómo está implementado.

## 9. Arquitectura: lo que nos quedó

*Presenta: Ricardo Fábrega*

**QUÉ DECIR**

Nos quedamos con cuatro ideas. Cada servicio es dueño de sus datos, y entre servicios viajan mensajes, no tablas.

La arquitectura hexagonal ordena el código por dentro y no cambia cómo se instala.

Y separar tiene un costo: más comunicación por red, más bases y más partes que pueden fallar. Por eso conviene usar microservicios cuando el negocio lo justifica. Para un sistema pequeño, una sola aplicación bien organizada puede ser la mejor decisión.

Ahora entraremos en la base de datos. Le paso la palabra a Marlene.

**SI SE TE OLVIDA, DI ESTO**

- Cada servicio controla sus propios datos.
- Entre los servicios viajan mensajes, no tablas.
- Separar servicios tiene un costo y debe justificarse.

**SI HAY TIEMPO**

- La parte más difícil de los microservicios fue definir las fronteras y decidir a quién pertenecía cada dato. Una vez resuelto eso, el resto del diseño se ordenó con mayor facilidad.

# 3 · Base de datos

Dos bases PostgreSQL, el ORM, los datos de conexión y las transacciones.

## 10. Dos bases, una por servicio

*Presenta: Marlene Bonilla*

**QUÉ DECIR**

Gracias, Ricardo. El sistema tiene dos bases de datos, una para cada servicio, y ambas utilizan PostgreSQL, versión diecisiete.

Usamos dos porque cada servicio es dueño de sus datos. Solicitudes guarda los trámites y los eventos pendientes, mientras notificaciones registra los eventos que ya recibió.

Así, si una base cambia su estructura, la otra no se ve afectada. No hacemos consultas que combinen tablas de ambas bases ni definimos llaves foráneas entre ellas, es decir, referencias que obliguen a que un dato exista en la otra base.

Cuando un servicio necesita un dato del otro, lo pide por la red. Además, cada base funciona por separado y tiene su propio disco.

Por eso los datos permanecen cuando un servicio se reinicia y, si una base presenta una falla, la otra puede continuar trabajando.

Ahora veamos qué información guarda cada una.

**SI SE TE OLVIDA, DI ESTO**

- Tenemos dos bases PostgreSQL, una por servicio.
- No hacemos consultas que crucen las bases.
- Cada base conserva sus datos en su propio disco.

**SI PREGUNTAN**

**¿Y si necesito un reporte que junte datos de las dos?**
Se prepara fuera de los servicios, por ejemplo con una réplica o un proceso de reportes que lea ambas fuentes. Un servicio no debe entrar en la base del otro.

**SI PREGUNTAN**

**¿No es más difícil administrar dos bases?**
Son dos bases pequeñas y cada una tiene una responsabilidad. Se respaldan y vigilan como cualquier PostgreSQL, y la caída de una no detiene la otra.

**SI HAY TIEMPO**

- Tener una base por servicio no duplica necesariamente el trabajo de diseño. Cada base es pequeña, tiene pocas tablas y atiende una responsabilidad concreta.
- En Kubernetes, cada base utiliza un disco persistente. Si el contenedor se reinicia, los datos permanecen y solo se perderían si se eliminara ese disco.

---

## 11. La estructura de las bases

*Presenta: Marlene Bonilla*

**QUÉ DECIR**

Veamos qué hay dentro. En la base de solicitudes, la tabla solicitud guarda el número, el tipo y el estado del trámite.

La propia base solo acepta los cuatro estados válidos, incluso si alguien intenta escribir directamente en la tabla. Al lado está la tabla evento, que funciona como una bandeja de salida.

Cada fila registra un evento, con un número único y una marca que indica si está pendiente o ya fue entregado.

La base de notificaciones tiene una tabla principal con los eventos que ya recibió. Funciona como el libro de registro de la entrada de una oficina.

No existe una conexión directa entre las bases. Lo que las relaciona es el número del evento, que viaja en el mensaje entre los servicios.

Ahora veremos un cambio importante frente a muchas aplicaciones actuales: dónde vive la regla del negocio.

**SI SE TE OLVIDA, DI ESTO**

- Una tabla guarda la solicitud.
- Otra mantiene los eventos pendientes.
- Notificaciones registra los eventos que ya recibió.

**SI PREGUNTAN**

**¿Dónde se guarda el texto de cada aviso?**

En este proyecto queda en la memoria del servicio de notificaciones y la base solo registra los eventos procesados. En producción usaríamos una tabla adicional.

**SI PREGUNTAN**

**¿Por qué usan un identificador largo, llamado UUID, y no un número consecutivo?**

Porque la aplicación puede generarlo sin consultar la base, con una probabilidad de coincidencia extremadamente baja. Conservamos ese mismo identificador en cada reintento para reconocer el evento.

**SI HAY TIEMPO**

- La tabla evento tiene un índice que incluye únicamente las filas pendientes. Como las consultamos cada cinco segundos, el índice sigue siendo pequeño aunque crezca el historial.
- Las tablas se crean mediante un archivo que puede ejecutarse en cada arranque sin causar daños. Ese archivo también actualiza bases anteriores con los elementos que les falten.

---

## 12. Del procedimiento almacenado al ORM

*Presenta: Marlene Bonilla*

**QUÉ DECIR**

Aquí está uno de los cambios principales frente a muchas aplicaciones conocidas. La diferencia no es la base de datos, sino el lugar donde vive la regla del negocio.

En el esquema habitual, la persona pulsa Enviar, la aplicación llama a un procedimiento almacenado y ese procedimiento decide y guarda. Por lo tanto, la regla vive en la base.

En ARKA-Lite, la regla vive en el servicio junto con su prueba. Cuando la regla permite la operación, una herramienta llamada ORM prepara las instrucciones para guardar los datos. Esas instrucciones se escriben en SQL, el lenguaje con el que consultamos y modificamos la información de la base.

Esto no significa que los procedimientos estén mal ni que la base pierda importancia: sigue cuidando las llaves, las restricciones y los índices. Solo movimos la decisión a otro lugar.

Jossett lo mostrará con un ejemplo y después explicará qué es un ORM.

Jossett, por favor.

**SI SE TE OLVIDA, DI ESTO**

- La diferencia está en dónde vive la regla.
- En ARKA-Lite vive en el servicio y tiene su prueba.
- La base sigue protegiendo llaves, restricciones e índices.

**SI PREGUNTAN**

**¿Y el rendimiento? ¿El SQL que genera es bueno?**

Sí, para guardar y buscar por llave. En consultas complejas podemos escribir SQL propio, mientras la base sigue manejando índices y restricciones.

**SI PREGUNTAN**

**¿Hay que migrar todos los procedimientos?**

No. Podemos comenzar con las reglas que cambian con frecuencia, mientras los reportes y procesos masivos permanecen en la base.

**SI HAY TIEMPO**

- Los procedimientos siguen siendo adecuados para reportes y procesos masivos. Lo que cambia es dónde colocamos las reglas que modificamos y probamos con mayor frecuencia.
- Cuando la regla está en el código, cada cambio pasa por revisión, pruebas automáticas y el pipeline, como cualquier otra parte del sistema.

---

## 13. La misma regla, escrita de las dos formas

*Presenta: Jossett Franceschi*

**QUÉ DECIR**

Gracias, Marlene. Veamos esa diferencia con una de las reglas que mencionó Luis: una solicitud solo puede enviarse cuando está en borrador.

A la izquierda, el procedimiento consulta el estado, valida la condición y actualiza el dato. No necesitamos leer cada línea para notar que la regla y el dato están juntos en la base.

A la derecha aparece la misma regla en el código de la aplicación. Si el estado no es borrador, se produce un error. Si lo es, la solicitud pasa a enviada.

La regla no cambia; cambia dónde se ejecuta. Y al tenerla en el código, podemos probarla en milésimas de segundo, sin una base.

Veamos ahora qué es exactamente un ORM.

**SI SE TE OLVIDA, DI ESTO**

- Estamos comparando responsabilidades, no tecnologías.
- La misma regla puede vivir en la base o en la aplicación.
- En el código podemos probarla rápidamente y sin una base.

**SI PREGUNTAN**

**¿Y si la regla necesita consultar muchos datos?**

El servicio pide por llave únicamente los datos necesarios para decidir. Si el cálculo abarca millones de filas, puede seguir en una consulta o un procedimiento bien delimitado.

**SI HAY TIEMPO**

- Para probar esta regla creamos una solicitud ya enviada, intentamos enviarla otra vez y verificamos el rechazo. La prueba tarda milésimas de segundo y no necesita una base.
- Con un procedimiento habría que preparar los datos, ejecutarlo y revisar el resultado en una base. También funciona, pero resulta más lento de repetir con cada cambio.
- El objetivo del proyecto es conocer otro enfoque y evaluar cuándo conviene usarlo en desarrollos futuros, sin reemplazar lo que ya funciona.

## 14. ¿Qué es un ORM?

*Presenta: Jossett Franceschi*

**QUÉ DECIR**

ORM significa mapeo objeto-relacional. En palabras sencillas, es una herramienta que hace de intérprete entre la aplicación y la base de datos.

La aplicación trabaja con objetos, como una solicitud que tiene número, tipo y estado. La base, en cambio, trabaja con tablas, filas y columnas.

El ORM traduce de un lado a otro. Cuando la aplicación le pide guardar una solicitud, identifica la tabla y la columna de cada dato, genera el SQL y lo envía.

Cuando la base responde, hace el recorrido contrario y convierte la fila en un objeto. Es como un intérprete en una reunión, donde cada parte puede hablar su propio idioma.

En nuestro proyecto usamos Hibernate, aunque la idea existe en otras tecnologías. Por ejemplo, .NET tiene Entity Framework.

El ORM no diseña la base por nosotros: las llaves, las relaciones y los índices los seguimos definiendo. Y si una consulta es compleja, podemos escribirla a mano.

Veamos ahora cómo una clase de la aplicación se relaciona con una tabla.

**SI SE TE OLVIDA, DI ESTO**

- El ORM traduce entre objetos y tablas.
- Hibernate genera el SQL de las operaciones comunes.
- Nosotros seguimos diseñando la base y podemos escribir SQL a mano.

**SI PREGUNTAN**

**¿El ORM reemplaza a quien administra la base de datos?**

No. Esa persona sigue diseñando el modelo, cuidando los índices y revisando el SQL generado, aunque la herramienta resuelva muchas consultas rutinarias.

**SI HAY TIEMPO**

- En Java, JPA define el estándar y Hibernate es una herramienta que lo implementa. Podemos verlo como una norma y un producto que cumple esa norma.
- Cuando la herramienta escribe el SQL rutinario, reducimos el código repetitivo y usamos los parámetros de forma consistente.
- Un ORM también puede generar consultas ineficientes si nadie las revisa. Por eso seguimos necesitando personas que entiendan el funcionamiento de las bases de datos.

## 15. El ORM por dentro: una clase es una tabla

*Presenta: Jossett Franceschi*

**QUÉ DECIR**

Aquí vemos la traducción lado a lado. A la izquierda está una clase de Java y, a la derecha, la tabla solicitud.

Una marca le indica al ORM que esa clase representa la tabla. Después, cada campo corresponde a una columna y la marca Id identifica la llave primaria.

Con ese mapeo, el ORM genera las operaciones básicas. Para guardar una solicitud nueva, el ORM genera un INSERT, que agrega una fila. Para buscarla por número, genera un SELECT, que consulta los datos.

Cuando enviamos una solicitud existente, genera un UPDATE, que actualiza su estado. Los valores viajan como parámetros, representados por los signos de interrogación, y nunca se pegan al texto del SQL.

Es el mismo principio utilizado con los parámetros de un procedimiento: los valores se tratan como datos y no como instrucciones. Así evitamos que un texto ingresado por una persona altere la consulta, lo que se conoce como inyección de SQL.

Así cada pieza hace una sola cosa: las reglas deciden, el ORM traduce y la base guarda. Si algo falla, sabemos en qué punto buscar.

Para llegar a la base, el ORM necesita su dirección, usuario y clave. Veamos dónde guardamos esos datos.

**SI SE TE OLVIDA, DI ESTO**

- Cada campo de la clase corresponde a una columna de la tabla.
- El ORM genera las instrucciones para agregar, consultar y actualizar los datos.
- Los valores viajan como parámetros seguros.

**SI PREGUNTAN**

**¿Se puede ver el SQL que genera?**

Sí. Una línea de configuración permite mostrarlo en los registros, lo que ayuda a revisar su funcionamiento y rendimiento.

**SI HAY TIEMPO**

- El repositorio del proyecto no contiene consultas escritas para las operaciones básicas. Al heredar de una interfaz, obtiene las funciones de guardar, buscar, listar y borrar.
- Al guardar, el ORM utiliza la llave para determinar si corresponde un INSERT o un UPDATE. Por eso es importante definir correctamente las llaves.

---

## 16. ¿Dónde guardamos los datos de conexión?

*Presenta: Jossett Franceschi*

**QUÉ DECIR**

Toda aplicación necesita conocer la ubicación de la base, el usuario y la clave para entrar. A este conjunto de datos lo llamamos cadena de conexión.

En nuestros sistemas actuales la guardamos en un archivo web.config dentro del servidor. Esa ya es una buena práctica, porque la información no está escrita en el código.

ARKA-Lite aplica la misma idea con otra herramienta. En lugar de usar un archivo junto a la aplicación, recibe variables de entorno cuando arranca.

Así, el ambiente donde se ejecuta le entrega la dirección, el usuario y la clave. La misma aplicación puede funcionar en desarrollo, pruebas y producción sin cambiar su código.

Cada ambiente proporciona sus propios valores. Por eso, si cambia el servidor o renovamos una clave, solo actualizamos la configuración del ambiente.

Ahora Marlene explicará cómo una transacción garantiza que los datos queden completos. Marlene, adelante.

**SI SE TE OLVIDA, DI ESTO**

- La conexión se configura fuera del código.
- Aquí usamos variables que entrega el ambiente.
- La misma aplicación sirve en distintos ambientes.

**SI PREGUNTAN**

**¿Dónde queda la clave en Kubernetes?**

Se guarda en un objeto para datos sensibles llamado Secret, que el contenedor recibe al arrancar. Los permisos controlan quién puede leerlo.

**SI PREGUNTAN**

**¿Hay que dejar de usar web.config?**

No. Sigue siendo válido para las aplicaciones que funcionan de esa manera. Lo importante es mantener la clave fuera del código y del repositorio.

**SI HAY TIEMPO**

- En Kubernetes, el Secret guarda la clave y las demás variables contienen el resto de la configuración.
- El proyecto tiene valores predeterminados para trabajar en una computadora. En un ambiente real no los usamos, porque la configuración siempre llega desde el entorno.
- En producción también podemos guardar las claves en una bóveda especializada. La idea se mantiene igual, porque permanecen fuera del código.

---

## 17. Una transacción: todo o nada

*Presenta: Marlene Bonilla*

**QUÉ DECIR**

Gracias, Jossett. Este concepto es conocido para quienes trabajan con bases de datos, aunque aquí lo controlamos desde la aplicación.

Cuando enviamos una solicitud, abrimos una transacción: un grupo de cambios que se guardan juntos o se deshacen completos. Ese inicio corresponde al BEGIN. Primero cambiamos el estado a enviada y, dentro de la misma transacción, registramos el evento pendiente.

Al final, el COMMIT guarda ambos cambios juntos. Si algo falla en el proceso, hacemos ROLLBACK y deshacemos todo.

Funciona como una transferencia bancaria. El dinero debe salir de una cuenta y entrar en la otra, o no debe ocurrir ningún cambio.

En nuestro caso, el cambio a enviada y el evento pendiente quedan guardados juntos. La transacción evita que se conserve uno sin el otro. Esa garantía sostiene la interconexión que veremos a continuación.

Antes de pasar a esa interconexión, resumamos lo que aprendimos sobre la base de datos.

**SI SE TE OLVIDA, DI ESTO**

- El cambio de estado y el evento pendiente se guardan juntos.
- Si algo falla, se deshacen ambos cambios.
- La operación nunca queda a medias.

**SI PREGUNTAN**

**¿Esto reemplaza las transacciones dentro de un procedimiento?**

Es el mismo concepto de BEGIN, COMMIT y ROLLBACK. La diferencia es que la aplicación abre la transacción alrededor de toda la operación.

**SI HAY TIEMPO**

- La transacción cubre toda la operación de envío. Si falla el INSERT del aviso, por ejemplo debido a una restricción, también se deshace el UPDATE de la solicitud.
- Preferimos una transacción local dentro de una base, porque resulta más confiable que intentar coordinar dos sistemas al mismo tiempo.

---

## 18. Base de datos: lo que aprendimos

*Presenta: Marlene Bonilla*

**QUÉ DECIR**

De la base de datos nos llevamos cuatro ideas. El ORM genera el SQL rutinario, pero no elimina el trabajo de diseñar llaves, restricciones e índices.

También mantenemos los datos de conexión fuera del código y los entregamos desde el ambiente. La transacción continúa siendo una de las herramientas más confiables para proteger una operación.

Además, guardamos y controlamos la estructura de la base junto con el código. Para quienes trabajamos con bases de datos, el mensaje es claro: ese conocimiento sigue siendo fundamental.

Ahora veremos cómo los servicios se comunican sin perder ni duplicar información. Le paso la palabra a Abdiel.

**SI SE TE OLVIDA, DI ESTO**

- El ORM ayuda, pero no reemplaza el diseño de la base.
- La conexión queda fuera del código.
- Las transacciones siguen protegiendo los datos.

**SI HAY TIEMPO**

- El rol de base de datos no desaparece. Se concentra más en diseñar el modelo, cuidar los índices y restricciones, y revisar el SQL generado por el ORM.

# 4 · Interconexión de servicios

Cómo se hablan los servicios: síncrono, asíncrono, Outbox e idempotencia.

## 19. Síncrono o asíncrono

*Presenta: Abdiel Barria*

**QUÉ DECIR**

Gracias, Marlene. Como los servicios no comparten una base, necesitamos definir cómo se comunican, y hay dos formas de hacerlo. La síncrona se parece a una llamada telefónica, porque hacemos el pedido y esperamos la respuesta.

La usamos para crear, aprobar o rechazar una solicitud, ya que la persona necesita conocer el resultado de inmediato. Si el otro lado está caído, verá el error.

La comunicación asíncrona se parece a un correo. Enviamos el mensaje y seguimos trabajando sin esperar, aunque la respuesta pueda tardar unos segundos.

En nuestro sistema, si el otro servicio está caído, el mensaje queda guardado hasta que podamos entregarlo. En un pago con tarjeta, por ejemplo, la aprobación es síncrona porque necesitamos saber si fue aceptado.

En cambio, el comprobante por correo es asíncrono porque puede llegar después. La regla es sencilla. Usamos comunicación síncrona cuando necesitamos conocer el resultado para continuar, y asíncrona cuando el trabajo puede completarse después.

Ahora veamos cómo hacemos que ese mensaje espere sin perderse mediante el patrón Outbox.

**SI SE TE OLVIDA, DI ESTO**

- Síncrono significa pedir y esperar.
- Asíncrono significa enviar y continuar.
- Elegimos según cuándo se necesita la respuesta.

**SI PREGUNTAN**

**¿Por qué no hacerlo todo asíncrono?**

Porque quien crea o aprueba una solicitud necesita saber en ese momento si la operación funcionó. Reservamos lo asíncrono para lo que puede esperar.

**SI HAY TIEMPO**

- En cada caso preguntamos si la persona debe enterarse cuando el otro sistema está caído. Si la respuesta es no, podemos manejarlo de forma asíncrona.
- Esto también ocurre en los sistemas actuales. Un procedimiento que llama a otro sistema y espera su respuesta falla cuando ese sistema no está disponible.

---

## 20. Patrón Outbox: guardar primero, avisar después

*Presenta: Abdiel Barria*

**QUÉ DECIR**

Si al enviar una solicitud llamáramos directamente a notificaciones y el servicio estuviera caído, la solicitud podría quedar enviada sin que nadie recibiera el aviso.

Para resolverlo usamos el patrón Outbox, que significa bandeja de salida. Dentro de una sola transacción guardamos la solicitud y un evento pendiente, que es el registro de lo que falta comunicar.

La persona recibe la respuesta de inmediato, mientras el evento queda esperando como una carta que todavía no ha salido.

Después, un proceso llamado relevador revisa la bandeja cada cinco segundos y entrega los eventos pendientes. Podemos verlo como el mensajero del sistema.

Si la entrega funciona, marca el evento como entregado. Si falla, lo deja pendiente y vuelve a intentarlo en la siguiente revisión.

Así, aunque notificaciones se detenga, ningún evento se pierde y todos se entregan cuando regresa.

La misma idea puede aplicarse con procedimientos almacenados: guardamos el evento en una tabla de pendientes y una tarea programada se encarga de entregarlo.

Como el sistema reintenta las entregas, ahora debemos evitar que el mismo evento produzca dos avisos.

**SI SE TE OLVIDA, DI ESTO**

- Primero guardamos y después avisamos.
- El relevador revisa la bandeja cada cinco segundos.
- Si la entrega falla, el evento permanece guardado.
- La misma idea funciona con una tabla de pendientes y una tarea programada.

**SI PREGUNTAN**

**¿Que el envío responda bien significa que el aviso ya existe?**

No. Significa que la solicitud y el evento quedaron guardados. El aviso se crea después.

**SI PREGUNTAN**

**¿Qué pasa si se cae solicitudes con eventos pendientes?**

Los eventos permanecen en la base. Cuando el servicio regresa, el relevador los encuentra y los entrega.

**SI HAY TIEMPO**

- El relevador intenta entregar cada evento hasta tres veces por vuelta y espera un poco más entre intentos. Si todos fallan, la fila permanece pendiente y vuelve a probar cinco segundos después.
- El número del evento se genera una sola vez al guardar. Por eso cada reintento lleva el mismo número y el receptor puede reconocerlo.
- Con procedimientos, la receta incluye una tabla de pendientes, una columna procesado, un INSERT dentro de la transacción y un job que recorre las filas.

## 21. Idempotencia: el mismo evento, un solo aviso

*Presenta: Abdiel Barria*

**QUÉ DECIR**

Un evento puede llegar dos veces. Por ejemplo, notificaciones lo recibe correctamente, pero la confirmación se pierde en la red. Como solicitudes no sabe que llegó, vuelve a enviarlo.

Para manejarlo usamos la idempotencia, una palabra técnica para una idea sencilla. Significa que repetir la misma operación no cambia el resultado.

Cada evento trae un número fijo, que llamamos el número del evento. Cuando llega, notificaciones lo busca en la lista. Si es nuevo, crea el aviso y registra el número. Si ya estaba registrado, lo ignora.

Es como un portero con una lista de invitados, porque quien ya entró no puede entrar una segunda vez.

Las dos soluciones se complementan. La bandeja de salida evita perder eventos y la idempotencia impide duplicar avisos.

Este principio sirve para cualquier proceso que tenga reintentos, como un pago, un correo o una importación de archivos.

Con ambas garantías claras, veamos en una sola imagen quién habla con quién, y cómo.

**SI SE TE OLVIDA, DI ESTO**

- Idempotencia significa repetir sin cambiar el resultado.
- El número del evento permite reconocer duplicados.
- La bandeja de salida conserva el evento y la idempotencia evita que una entrega repetida cree otro aviso.

**SI PREGUNTAN**

**¿Qué resuelve el Outbox y qué resuelve la idempotencia?**

Outbox evita perder el evento. La idempotencia evita que ese evento produzca el mismo efecto dos veces.

**SI PREGUNTAN**

**¿Y si llegan dos entregas al mismo tiempo?**

La llave primaria permite registrar el número una sola vez. La otra entrega falla y, al reintentarse, encuentra el evento ya procesado.

**SI HAY TIEMPO**

- Lo probamos desde la pantalla al volver a entregar un evento ya recibido. Aunque repetimos la entrega, el contador de avisos duplicados permanece en cero.
- La receta sirve en cualquier sistema. Necesitamos un número único por operación y una tabla que recuerde cuáles ya fueron procesadas.

## 22. Resumen: quién habla con quién, y cómo

*Presenta: Abdiel Barria*

**QUÉ DECIR**

Con las piezas claras, veamos el recorrido completo en una sola imagen. Es el resumen de esta parte.

Arriba está lo que ocurre mientras la persona espera. El navegador solo conoce la pantalla, y la pantalla pasa cada pedido por un intermediario que lo lleva al servicio de solicitudes. Es como la recepción de una empresa: pedimos un departamento y nos conectan con él.

Solicitudes aplica la regla, guarda la solicitud y su evento en la misma transacción y responde de inmediato. Esa es la parte síncrona.

Abajo está lo que pasa después, sin que nadie espere. El relevador toma el evento de la bandeja de salida y se lo entrega a notificaciones, que revisa el número, crea un solo aviso y anota que ya lo recibió. Esa es la parte asíncrona.

Dos detalles unen todo. Las piezas se encuentran por su nombre y no por una dirección fija, y cada llamada lleva un número de seguimiento, distinto del número del evento, que nos permite reconstruir el recorrido en los registros. Moises lo retomará en la observabilidad.

Con el recorrido completo a la vista, resumamos lo aprendido sobre la interconexión.

**SI SE TE OLVIDA, DI ESTO**

- Arriba, lo que pasa mientras la persona espera.
- Abajo, lo que pasa después, por su cuenta.
- Se encuentran por nombre y un número sigue cada llamada.

**QUÉ HACER (NO SE DICE)**

- Recorrer la lámina de izquierda a derecha: primero el camino de arriba y después el de abajo.

**SI PREGUNTAN**

**¿Por qué no una cola de mensajes como RabbitMQ o Kafka?**

Para este proyecto, basta con guardar el evento pendiente y entregarlo después mediante una llamada web al otro servicio. No necesitamos agregar una herramienta de mensajería. Si aumenta el volumen, podemos llevar el mismo diseño a una cola sin cambiar las reglas.

**SI HAY TIEMPO**

- La llamada entre servicios utiliza un cliente preparado para conservar el número de seguimiento. Con un cliente común se perdería esa continuidad, por eso lo definimos como una regla del equipo.
- Los servicios se comunican por HTTP y envían mensajes en JSON, un formato de texto que pueden leer tanto las personas como los sistemas.

## 23. Interconexión: lo que aprendimos

*Presenta: Abdiel Barria*

**QUÉ DECIR**

De la interconexión nos llevamos cuatro ideas. Esperamos la respuesta cuando la necesitamos para continuar y dejamos para después el trabajo que puede esperar.

También aprendimos que guardar primero y avisar después es más confiable que coordinar dos sistemas al mismo tiempo.

Cuando una operación se reintenta, el receptor debe reconocer lo repetido. Además, los servicios solo comparten el formato del mensaje, así que cualquier cambio debe coordinarse en ambos lados.

Estas decisiones no dependen de Java y pueden aplicarse a los sistemas que ya tenemos.

Ahora veremos cómo llevamos el código y los datos hasta un servidor. Le paso la palabra a Kevin, de infraestructura.

**SI SE TE OLVIDA, DI ESTO**

- Elegimos entre esperar o avisar después.
- Guardamos primero para no perder información.
- Reconocemos repetidos para evitar duplicados.

**SI HAY TIEMPO**

- Juntas, la bandeja de salida y la idempotencia permiten que un mensaje llegue varias veces, pero produzca su efecto una sola vez. Esta es una base común en sistemas de pagos y notificaciones.

# 5 · DevOps: Docker y Kubernetes

Docker empaqueta, el pipeline prueba cada cambio y Kubernetes administra.

## 24. ¿Qué es DevOps? Del código a la operación

*Presenta: Kevin*

**QUÉ DECIR**

Gracias, Abdiel. DevOps no es una herramienta ni un puesto específico. Es una forma de trabajar en la que desarrollo y operación colaboran y cada cambio sigue un camino repetible.

Podemos imaginarlo como una línea de producción. Cada producto pasa por las mismas estaciones, en el mismo orden, y ninguna revisión se omite.

Primero escribimos y revisamos el cambio. Después lo probamos, lo empaquetamos, lo publicamos, lo instalamos y lo operamos en el servidor.

También vigilamos el sistema mediante sus registros. Lo que aprendemos durante la operación vuelve al código y el ciclo comienza otra vez.

En esta parte, yo explicaré Docker, Jossett mostrará el camino automático de los cambios y Moises explicará Kubernetes.

Comencemos por entender cómo se complementan Docker y Kubernetes.

**SI SE TE OLVIDA, DI ESTO**

- DevOps es una forma de trabajar.
- Cada cambio sigue un camino repetible.
- El ciclo une desarrollo, entrega y operación.

**SI PREGUNTAN**

**¿DevOps es un equipo aparte?**

Es una práctica. Puede haber personas que la impulsen, pero funciona cuando desarrollo e infraestructura comparten el proceso y las herramientas.

**SI HAY TIEMPO**

- En ARKA-Lite utilizamos Git y GitHub para el código, pruebas automáticas para validarlo, Docker para empaquetarlo, Kubernetes para operarlo y Grafana para observarlo.
- La mayoría de los pasos ya no depende de la memoria de una persona. Si uno falla, el proceso se detiene y muestra con claridad dónde ocurrió el problema.

---

## 25. Docker y Kubernetes: no compiten, se complementan

*Presenta: Kevin*

**QUÉ DECIR**

Docker y Kubernetes se escuchan juntos con frecuencia, y a veces se presentan como alternativas. En realidad cumplen funciones distintas y se complementan.

Docker empaqueta la aplicación con todo lo necesario para ejecutarla, dentro de una imagen que funciona igual en una computadora o en un servidor. Así evitamos que algo funcione en una máquina y falle en otra.

Kubernetes comienza donde termina Docker: decide dónde corren esas aplicaciones, las vigila y reemplaza las que fallan.

Pensemos en el transporte de carga que conocemos en Panamá. Docker es el contenedor estándar, que se mueve igual sin importar lo que lleva, y Kubernetes es el puerto que los acomoda, los vigila y reemplaza el que se daña.

Ahora veamos cómo una receta termina convertida en un contenedor.

**SI SE TE OLVIDA, DI ESTO**

- Docker empaqueta y Kubernetes administra.
- La imagen funciona igual en distintos ambientes.
- Docker es el contenedor y Kubernetes es el puerto.

**SI PREGUNTAN**

**¿Un contenedor de Docker es una máquina virtual?**

No. Es más liviano porque comparte el núcleo del sistema operativo y aísla la aplicación y sus dependencias.

**SI PREGUNTAN**

**¿Necesitamos Kubernetes para todo?**

No. Para un sistema pequeño puede bastar Docker Compose, una herramienta que permite levantar varios contenedores juntos. La veremos en unas láminas. Kubernetes cobra valor cuando hay varios servicios, múltiples copias o recuperación automática.

**SI HAY TIEMPO**

- La comparación funciona porque el contenedor de carga también estandarizó la forma de mover distintos productos. La misma caja puede viajar en barco, tren o camión sin importar su contenido.
- Podemos usar Docker sin Kubernetes en un servidor pequeño. También Kubernetes puede ejecutar imágenes creadas con otras herramientas, aunque juntos cubren todo el recorrido.

---

## 26. De código a contenedor

*Presenta: Kevin*

**QUÉ DECIR**

Un contenedor nace a partir de tres conceptos. Todo comienza con una receta llamada Dockerfile, donde indicamos qué se instala y qué archivos se copian.

Con esa receta construimos una imagen, que funciona como un molde. Contiene la aplicación y únicamente lo necesario para ejecutarla.

La imagen no cambia, así que instalamos exactamente la misma que probamos. Cuando ponemos esa imagen a funcionar, obtenemos un contenedor.

Del mismo molde pueden salir varios contenedores iguales. Además, el nuestro se ejecuta con un usuario sin privilegios, por seguridad: si alguien abusara de la aplicación, tendría muy poco margen para hacer daño.

Por eso podemos reemplazar un contenedor y crear otro a partir de la misma imagen, sin reconstruir la aplicación.

Con la receta, el molde y el contenedor claros, veamos qué ventajas nos ofrece Docker.

**SI SE TE OLVIDA, DI ESTO**

- El Dockerfile es la receta.
- La imagen es el molde.
- El contenedor es la imagen en ejecución, sin privilegios.

**SI PREGUNTAN**

**¿Dónde se guardan las imágenes?**

Se guardan en un registro, que funciona como una biblioteca de imágenes. Nosotros usamos el registro de GitHub.

**SI HAY TIEMPO**

- Nuestra receta tiene dos etapas. Una compila con las herramientas de desarrollo y la otra conserva únicamente lo necesario para ejecutar, sin incluir el código fuente ni las herramientas de compilación.
- Las imágenes se organizan por capas. Cuando solo cambia el código, reutilizamos las capas anteriores y reducimos el tiempo de construcción y descarga.

---

## 27. Ventajas de Docker

*Presenta: Kevin*

**QUÉ DECIR**

Docker nos ofrece varias ventajas. Primero, la misma imagen funciona en la computadora, en pruebas y en producción.

También es liviano, porque un contenedor arranca en segundos y no necesita cargar un sistema operativo completo como una máquina virtual.

Además, cada aplicación mantiene aisladas sus dependencias. Una puede usar Java 26 y otra Node en el mismo servidor sin interferir entre sí.

Podemos limitar la memoria y la capacidad de procesamiento, o CPU, que utiliza cada contenedor. Así, una aplicación que se descontrola no consume todos los recursos ni afecta a las demás.

Las imágenes también tienen versiones, de modo que podemos volver atrás utilizando una anterior. Finalmente, levantamos el sistema con un comando en lugar de seguir una lista manual.

Ese comando no levanta una sola pieza, sino el sistema completo.

**SI SE TE OLVIDA, DI ESTO**

- La misma imagen funciona en todos los ambientes.
- Cada aplicación mantiene aisladas sus dependencias.
- Podemos limitar recursos y volver a una versión anterior.

**SI PREGUNTAN**

**¿En ARKA-Lite tienen tope de memoria y CPU en Docker?**

No lo configuramos en Compose porque se usa para trabajar en una computadora. En Kubernetes sí lo aplicamos a la pantalla.

**SI HAY TIEMPO**

- Una máquina virtual carga un sistema operativo completo, mientras un contenedor comparte el del servidor y lleva solo sus elementos. Por eso caben más contenedores en el mismo servidor.
- Los límites se definen al arrancar e indican cuánta memoria y CPU puede utilizar la aplicación. Sin ellos, un contenedor podría consumir todos los recursos disponibles.
- El aislamiento no reemplaza las demás medidas de seguridad. Por eso nuestro contenedor también funciona con un usuario sin privilegios.

---

## 28. Todo el sistema con un comando

*Presenta: Kevin*

**QUÉ DECIR**

Un solo contenedor no representa todo el sistema. Con Docker Compose levantamos mediante un comando los dos servicios, las dos bases, la pantalla y las herramientas de registros.

En total se ejecutan ocho contenedores. Compose los conecta dentro de una red privada, donde cada uno encuentra a los demás por su nombre, como mencionó Abdiel.

También los inicia en orden. Primero levanta las bases y después cada servicio espera hasta que su base esté disponible.

Además, entrega la configuración desde afuera, incluidos los datos de conexión que explicó Jossett.

El beneficio es directo. Una persona nueva puede levantar el sistema con un comando, sin configurar cada componente por separado.

Con el sistema reproducible, veamos cómo un cambio llega hasta una imagen publicada. Le paso la palabra a Jossett.

**SI SE TE OLVIDA, DI ESTO**

- Un comando levanta ocho contenedores.
- Compose los conecta y los inicia en orden.
- También les entrega la configuración.

**SI PREGUNTAN**

**Dentro de un contenedor, ¿a dónde lleva localhost?**

Lleva al mismo contenedor, no a otro servicio. Por eso utilizamos el nombre del servicio.

**SI PREGUNTAN**

**¿Compose sirve para producción?**

Puede servir para algo pequeño en un solo servidor. Cuando necesitamos varios servidores o recuperación automática, conviene Kubernetes.

**SI HAY TIEMPO**

- Si una base no arranca, el servicio correspondiente queda esperando y Compose muestra la causa. Así el error aparece en el lugar correcto.
- Desde la computadora solo abrimos lo necesario. La base de solicitudes no tiene una entrada externa y únicamente los servicios internos pueden alcanzarla.
- Antes de continuar, las bases informan cuándo están listas para aceptar conexiones mediante una herramienta de PostgreSQL.

---

## 29. Git: el historial y las ramas

*Presenta: Jossett Franceschi*

**QUÉ DECIR**

Gracias, Kevin. Antes de hablar de CI/CD necesitamos dos ideas de Git, la herramienta con la que guardamos el código.

La primera es el historial. Git guarda cada avance como un punto en una línea de tiempo. Así sabemos qué cambió, quién lo hizo y cuándo. También podemos volver a un punto anterior.

La línea del centro se llama main. Es la versión oficial, la que se instala, y nadie trabaja directamente sobre ella.

La segunda idea es la rama. Para cada tarea, la rama nace de un punto de main. Es una copia de main tal como está en ese momento. Ahí guardamos los avances sin tocar la versión oficial.

Arriba está la rama de una tarea y abajo la de otra. Las dos avanzan al mismo tiempo, cada una por su lado, sin pisarse.

Cuando una tarea está lista, su rama vuelve a main con su cambio. A eso le llamamos merge, y es la flecha verde.

Es como el control de cambios de un documento compartido: cada quien trabaja en su copia y, al final, se integra al original.

Con esas dos ideas, veamos paso por paso cómo un cambio viaja de una rama hasta main.

**SI SE TE OLVIDA, DI ESTO**

- Git guarda el historial de cada avance.
- main es la versión oficial.
- La rama nace como copia de main.
- Cada tarea se trabaja en su rama y vuelve a main con un merge.

**SI PREGUNTAN**

**¿Qué pasa si dos personas cambian lo mismo?**

Git detecta el conflicto y avisa. El equipo decide cómo combinar los cambios antes de incorporarlos a main.

**SI HAY TIEMPO**

- Cada avance guardado se llama commit y tiene un código único. Con ese código se puede volver exactamente a ese punto.
- GitHub es el lugar donde el equipo guarda y comparte el código, y donde se hacen las revisiones.

---

## 30. Cómo entra un cambio, paso por paso

*Presenta: Jossett Franceschi*

**QUÉ DECIR**

Ahora veamos el camino de un cambio en cinco pasos. Primero, abrimos una rama para la tarea. Segundo, ahí escribimos el cambio y su prueba.

Tercero, hacemos push: subimos la rama a GitHub, donde el equipo comparte el código. Cuarto, abrimos un Pull Request para que otra persona revise el cambio. En ese momento, las pruebas corren solas.

Quinto, si otra persona aprueba el cambio y las pruebas pasan, hacemos merge, que es unir la rama a main. Solo entonces el cambio forma parte de la versión oficial.

Esas pruebas no las corre una persona. Las corre el pipeline. Veamos qué es.

**SI SE TE OLVIDA, DI ESTO**

- Rama, cambio, push, Pull Request y merge, en ese orden.
- Nada entra a main sin revisión y sin pruebas.

**SI PREGUNTAN**

**¿Quién revisa el Pull Request?**

Otra persona del equipo. Quien hizo el cambio no puede aprobar su propio cambio.

**SI HAY TIEMPO**

- Describimos cada cambio con un formato común, por ejemplo, fix seguido de impedir el envío de una solicitud aprobada. Así el historial funciona como una bitácora.
- La documentación de los servicios se genera desde el código mediante Swagger, lo que reduce el riesgo de que quede desactualizada.
- Nadie incorpora cambios directamente en la versión principal, incluida la persona que lidera el equipo.

---

## 31. CI/CD: el control de calidad automático

*Presenta: Jossett Franceschi*

**QUÉ DECIR**

CI/CD reúne dos prácticas que automatizan este camino: integración continua y entrega continua. Sus tareas forman un pipeline, es decir, una lista de pasos que se ejecuta sola. Es como el control de calidad de una fábrica: cada producto pasa por las mismas revisiones.

El pipeline se activa cuando ocurre un evento, es decir, una acción que lo pone en marcha. En ARKA-Lite responde a dos eventos. Cada uno sigue un camino distinto.

Arriba, el primer evento ocurre cuando alguien abre o actualiza un Pull Request. Ahí se ejecuta la integración continua, o CI. GitHub compila la aplicación y ejecuta las pruebas con una base de datos temporal. El resultado queda en verde o en rojo.

Abajo, el segundo evento ocurre cuando hacemos merge a main. El pipeline compila y prueba otra vez. Si todo pasa, ejecuta la entrega continua, o CD: publica una imagen, un paquete listo para instalar con el código del cambio.

Nadie tiene que recordar esos pasos. En nuestro proyecto, la instalación final aún se hace con un comando. El siguiente paso sería automatizarla con una aprobación previa.

Ahora veamos dónde se escriben esos eventos y pasos: en el archivo ci.yml.

**SI SE TE OLVIDA, DI ESTO**

- El pipeline reacciona a eventos.
- En el Pull Request: compila y prueba.
- En el merge a main: además publica la imagen.
- La instalación final todavía usa un comando.

**SI PREGUNTAN**

**¿Se puede condicionar a otros eventos?**

Sí. Por ejemplo, a un push en cualquier rama, a una hora programada o a una etiqueta de versión.

**SI PREGUNTAN**

**¿Así funciona hoy ARKA-Lite?**

La pantalla, sí: su imagen se publica solo al llegar a main. Los servicios todavía publican también durante el Pull Request; es un ajuste que tenemos pendiente.

**SI PREGUNTAN**

**¿Por qué no se instala solo?**

Porque Kubernetes funciona en nuestro ambiente local de laboratorio. En un ambiente real agregaríamos al pipeline un paso de instalación con aprobación.

**SI PREGUNTAN**

**¿Las pruebas usan la base real?**

No. GitHub crea una base de datos PostgreSQL temporal para cada ejecución y la elimina al terminar.

**SI HAY TIEMPO**

- El pipeline ejecuta al mismo tiempo un trabajo para solicitudes, otro para notificaciones y otro para la pantalla. Así termina antes y muestra con precisión qué pieza falló.
- Además de pasar las pruebas, el servicio de solicitudes arranca realmente y debe mostrar una solicitud

de ejemplo. Compilar no garantiza que una aplicación pueda iniciar.

- Entrega continua significa dejar la imagen lista, mientras despliegue continuo significa instalarla automáticamente. Nuestro proyecto llega hasta la entrega.
- Las condiciones se escriben en el mismo archivo del pipeline: en qué eventos corre y qué pasos se hacen solo en algunos de ellos, como publicar la imagen solo al llegar a main.
- Actualmente las imágenes de los servicios se publican durante la revisión y la de la pantalla al llegar a main. Ya identificamos esa diferencia como un ajuste pendiente.

---

## 32. El pipeline se escribe en un archivo: ci.yml

*Presenta: Jossett Franceschi*

**QUÉ DECIR**

Todo lo que acabamos de ver está escrito en un archivo de texto llamado ci.yml. Está en la carpeta .github/workflows, dentro del repositorio. GitHub lo lee automáticamente.

A la izquierda vemos cuándo se ejecuta. La palabra on enumera los eventos: abrir o actualizar un Pull Request, y hacer push a main, como ocurre al hacer merge.

A la derecha vemos qué hace. Primero indica que se ejecuta en un servidor de GitHub. Luego, steps enumera los pasos en orden: compilar y probar, y publicar la imagen.

La línea if agrega una condición: el paso de publicar la imagen solo se ejecuta cuando hay un push a main. Así se forman los dos caminos de la lámina anterior.

Como es un archivo más del proyecto, también pasa por un Pull Request y se revisa. Si alguien cambia el pipeline, queda registrado quién lo cambió y por qué.

Veamos ahora qué ocurre cuando una de esas revisiones detecta un problema.

**SI SE TE OLVIDA, DI ESTO**

- El pipeline se escribe en ci.yml.
- on dice cuándo corre.
- steps enumera los pasos. if indica una condición.
- El archivo se revisa como el código.

**QUÉ HACER (NO SE DICE)**

- Es una versión simplificada del archivo real: no hace falta leer cada línea, basta señalar on, steps e if.

**SI PREGUNTAN**

**¿Hay que saber programar para leerlo?**

Ayuda, pero no es indispensable. YAML es un formato de texto con nombres y valores, y cada paso indica en palabras qué hace.

**SI HAY TIEMPO**

- GitHub ofrece plantillas para casi cualquier tecnología: se parte de una y se adapta al proyecto.
- El nombre del archivo puede ser otro; lo que importa es que esté en la carpeta .github/workflows.

## 33. Cuando un cambio rompe algo

*Presenta: Jossett Franceschi*

**QUÉ DECIR**

Veamos qué pasa cuando un cambio rompe una regla. Supongamos que alguien permite por error enviar dos veces la misma solicitud.

Si todo se revisara a mano, quizá lo descubriríamos semanas después, cuando ya hubiera afectado a una persona usuaria.

Aquí el pipeline arranca solo. En pocos minutos, falla la prueba que protege esa regla. GitHub muestra qué falló y dónde buscar el problema.

La persona corrige el problema y el pipeline vuelve a correr. Si main exige las pruebas en verde, el error no entra.

La integración continua funciona como una red de seguridad: permite hacer cambios con mayor confianza.

Por eso cada regla necesita una prueba. Esa prueba es la alarma que se activa cuando alguien modifica el comportamiento esperado.

Cuando el cambio se aprueba y las pruebas pasan, el pipeline publica la imagen con una etiqueta.

**SI SE TE OLVIDA, DI ESTO**

- Una prueba detecta la regla rota en minutos.
- GitHub muestra dónde ocurrió el problema.
- El error no llega a la versión principal.

**SI PREGUNTAN**

**¿Y si alguien se salta el pipeline?**

Todo cambio entra mediante Pull Request. Además, GitHub puede impedir el merge mientras el pipeline esté en rojo.

**SI HAY TIEMPO**

- Podemos configurar GitHub para bloquear un Pull Request cuando alguna prueba falla. Es una regla de la plataforma y no requiere modificar el código.
- El pipeline publica imágenes mediante un permiso temporal que GitHub entrega a cada ejecución, sin almacenar una contraseña.

## 34. Cada imagen, con su etiqueta

*Presenta: Jossett Franceschi*

**QUÉ DECIR**

Cuando las pruebas pasan, obtenemos trazabilidad: podemos relacionar cada imagen publicada con el cambio que la generó.

La imagen lleva el código de ese cambio, como el número de lote de un medicamento. Si aparece un problema, sabemos exactamente de dónde vino.

Con ese código podemos buscar en GitHub qué cambió, quién lo propuso, quién lo revisó y cuáles pruebas pasaron.

Si una versión presenta problemas, podemos volver a la imagen anterior, que continúa almacenada.

Si el equipo de operaciones pregunta qué versión está instalada, damos la etiqueta exacta y consultamos su historial. En el laboratorio todavía construimos las imágenes en una computadora. En un ambiente real usaríamos la imagen publicada por el pipeline.

La imagen ya está publicada y rastreada. Ahora veamos dónde se ejecuta. Le paso la palabra a Moises, que nos explicará Kubernetes.

**SI SE TE OLVIDA, DI ESTO**

- Cada imagen se conecta con un cambio específico.
- Podemos saber quién lo revisó y qué pruebas pasó.
- Si falla, volvemos a la imagen anterior.

**SI PREGUNTAN**

**¿Por qué no usar la etiqueta latest?**

Porque latest cambia con cada publicación y no identifica el código. La etiqueta basada en el cambio siempre señala una versión concreta.

**SI HAY TIEMPO**

- Ese código se llama SHA y Git genera uno para cada cambio. Usarlo evita la duda sobre si la imagen que se ejecuta es la nueva o la anterior.
- El archivo de Kubernetes indica qué imagen debe utilizar. Para cambiar de versión, modificamos esa referencia y volvemos a aplicar el archivo.

---

## 35. Cómo trabaja Kubernetes

*Presenta: Moises*

**QUÉ DECIR**

Gracias, Jossett. Kubernetes requiere una forma distinta de pensar. No le damos instrucciones paso a paso, sino que describimos el estado que queremos mantener.

Funciona como un termostato. No le indicamos cuándo debe encender y apagar el aire, sino la temperatura que queremos, y el equipo hace los ajustes.

En Kubernetes escribimos en un archivo qué imagen debe usar, cuántas copias necesitamos y qué configuración corresponde.

Luego entregamos ese archivo mediante un comando y Kubernetes crea los elementos solicitados.

A partir de ahí, compara continuamente lo que existe con lo que pedimos. Si falta algo, lo crea y, si sobra, lo elimina.

Por eso el sistema puede repararse solo. Como el archivo se guarda con el código, la configuración tampoco depende de la memoria de una persona.

Ahora veamos dos conceptos nuevos. Un Pod es el espacio que Kubernetes le da a cada contenedor, y un Service es el nombre fijo para encontrarlo.

**SI SE TE OLVIDA, DI ESTO**

- Describimos el estado que queremos.
- Kubernetes funciona como un termostato.
- Compara el estado y corrige las diferencias.

**SI PREGUNTAN**

**¿Qué pasa si alguien cambia algo a mano?**

Depende de qué cambie. Si alguien elimina un Pod, Kubernetes crea otro para mantener la cantidad de copias. Si modifica la configuración, debemos reflejar ese cambio en el archivo, para que lo guardado coincida con lo que está funcionando.

**SI HAY TIEMPO**

- Los archivos de ARKA-Lite están en la carpeta k8s del repositorio, separados por componente. Para reconstruir el sistema en otro clúster, volvemos a aplicar esos archivos.
- Estos archivos se llaman manifiestos y utilizan un formato de texto llamado YAML. Un clúster es un grupo de servidores que trabajan juntos.

---

## 36. Contenedores y Pods: cómo se complementan

*Presenta: Moises*

**QUÉ DECIR**

Sigamos el diagrama de izquierda a derecha. La imagen es el molde que hizo Docker, y el Deployment es el pedido que le hacemos a Kubernetes: en este ejemplo, dos copias de esa imagen.

A la derecha está un servidor del clúster, que es el grupo de servidores donde corren las aplicaciones. Cada contenedor vive dentro de un Pod, que le da su dirección, su memoria y su CPU. Siguiendo con el puerto, el contenedor es la carga y el Pod, su espacio en el patio.

Arriba está el Service. Como los Pods se reemplazan y cambian de dirección, el Service mantiene un nombre fijo y reparte el trabajo entre las copias.

En ARKA-Lite usamos una copia de cada pieza. Para tener dos, basta con cambiar un número en su archivo.

Ya conocemos las piezas principales. Ahora veamos qué capacidades nos ofrece Kubernetes con ellas.

**SI SE TE OLVIDA, DI ESTO**

- La imagen es el molde y el contenedor la ejecución.
- El Pod aporta red, memoria y CPU.
- El Deployment mantiene la cantidad de copias.
- El Service ofrece un nombre fijo.

**SI PREGUNTAN**

**¿Un Pod puede tener más de un contenedor?**

Sí, cuando dos procesos deben vivir juntos, aunque lo normal es uno. Los nuestros tienen uno principal y otro temporal que espera la base.

**SI PREGUNTAN**

**¿Dónde corren los Pods?**

Corren en los servidores del clúster, llamados nodos. Kubernetes elige el nodo según los recursos disponibles.

**SI HAY TIEMPO**

- El Pod permite que Kubernetes trabaje con distintas tecnologías de contenedores y también agrupe procesos que necesitan compartir la misma dirección.
- En ARKA-Lite, un contenedor temporal espera a que la base responda antes de iniciar el principal. Así evitamos que el servicio falle y se reinicie repetidamente.
- Los Pods se reemplazan en lugar de repararse. Por eso los datos importantes permanecen en la base y en su disco, no dentro del Pod.

---

## 37. Las bondades de Kubernetes

*Presenta: Moises*

**QUÉ DECIR**

Kubernetes aporta capacidades que Docker por sí solo no ofrece. Si un Pod se cae, levanta otro. Si aumenta la carga, pedimos más copias cambiando un número, y el Service reparte el trabajo entre ellas.

También actualiza poco a poco, sin apagar todo, y permite volver a la versión anterior si algo falla.

Controla la memoria y la CPU de cada Pod con una reserva y un tope. Es como un presupuesto: la reserva es lo que apartamos al planificar y el tope es el máximo permitido. En ARKA-Lite lo aplicamos a la pantalla.

Por último, guarda las claves aparte, en un objeto llamado Secret, y los datos en un disco que sobrevive al reinicio del Pod. Todo queda escrito en archivos, sin depender de la memoria de una persona.

Veamos qué ocurre cuando una de esas copias se cae.

**SI SE TE OLVIDA, DI ESTO**

- Kubernetes reemplaza los Pods que fallan.
- Puede aumentar copias y repartir el trabajo.
- La reserva es lo que apartamos y el tope, el máximo.
- Las claves y los datos van aparte.

**SI PREGUNTAN**

**¿Por qué la base no corre en un servicio administrado?**

Porque este es un laboratorio. En producción evaluaríamos una base administrada o una configuración de Kubernetes preparada para conservar la identidad y el almacenamiento de la base, junto con sus respaldos.

**SI PREGUNTAN**

**¿Qué número se pone de reserva y de tope?**

Comenzamos con una estimación, medimos el sistema y ajustamos. Un tope de memoria muy bajo provoca reinicios, y uno demasiado alto deja que un contenedor consuma más de lo conveniente.

**SI HAY TIEMPO**

- Las revisiones de salud indican cuándo un servicio está listo para trabajar. Un proceso puede haber arrancado sin estar todavía preparado para recibir pedidos.
- Podemos aumentar las copias porque los servicios no guardan el estado importante en memoria, sino en la base.
- En el laboratorio abrimos un túnel mediante un comando para llegar a la pantalla. En un ambiente real se expondría de forma permanente.
- La pantalla reserva veinticinco milésimas de CPU y treinta y dos megas de memoria, con topes de doscientas cincuenta milésimas y ciento veintiocho megas. Llegar al tope de CPU la hace más lenta; pasarse de memoria provoca un reinicio.
- Definir la reserva y el tope también permite calcular cuántos servidores necesitamos, en lugar de estimarlo sin mediciones.

## 38. Cuando un Pod se cae

*Presenta: Moises*

**QUÉ DECIR**

Supongamos que el servicio de solicitudes se cae por un error o por el reinicio del servidor. Kubernetes detecta que falta una copia y crea otra con la misma imagen.

La nueva copia espera hasta que su base esté lista. Después pasa la revisión de salud y comienza a recibir trabajo otra vez, sin que una persona tenga que intervenir.

Como en el laboratorio mantenemos una sola copia, tenemos unos segundos sin servicio. Con dos copias, la interrupción casi no se notaría.

Mientras tanto, los eventos pendientes permanecen guardados en la base y no se pierde información.

Aquí se conectan tres partes de la presentación. La base guarda, la bandeja de salida espera y Kubernetes repara.

Con este ejemplo cerramos la explicación de Kubernetes y resumimos lo aprendido de DevOps.

**SI SE TE OLVIDA, DI ESTO**

- Kubernetes reemplaza una copia que se cae.
- La nueva copia espera a que la base esté lista y pasa su revisión.
- Los eventos pendientes permanecen guardados.

**SI PREGUNTAN**

**¿Y si se cae la base?**

Su Pod también se recrea y los datos permanecen en el disco. Los servicios esperan hasta que la base regrese.

**SI HAY TIEMPO**

- Podemos demostrarlo eliminando el Pod mediante un comando. En la lista aparece uno nuevo y, después de unos segundos, vuelve a estar listo.
- Si se cae la base, ocurre algo similar. Kubernetes recrea el Pod y los datos continúan en su disco.

## 39. DevOps: lo que aprendimos

*Presenta: Moises*

**QUÉ DECIR**

De DevOps nos llevamos varias ideas. Docker y Kubernetes se complementan: uno empaqueta la aplicación y el otro la ejecuta y la cuida.

La imagen es el entregable que construimos una vez y reutilizamos entre ambientes. Además, la configuración pertenece al ambiente y no al código.

También aprendimos a describir el estado que queremos y a comprobar que los servicios realmente arrancan, en lugar de verificar solamente que compilan.

El mayor valor es que todo quedó escrito. Tenemos la receta de cada imagen, el pipeline y los archivos de Kubernetes, sin depender de la memoria de una persona.

El sistema ya está instalado. Ahora veamos cómo sabemos qué ocurre dentro.

**SI SE TE OLVIDA, DI ESTO**

- Docker empaqueta y Kubernetes opera.
- La misma imagen sirve en varios ambientes.
- La instalación queda escrita y puede repetirse.

**SI HAY TIEMPO**

- Docker y Kubernetes requieren aprendizaje y no todos los sistemas los necesitan. Sin embargo, siempre conviene documentar la instalación de una forma escrita y repetible.

# 6 · Observabilidad

Cómo sabemos qué pasa adentro: salud, registros y un número de seguimiento.

## 40. Observabilidad: tres preguntas

*Presenta: Moises*

**QUÉ DECIR**

Observabilidad es el nombre que damos a la capacidad de entender qué ocurre dentro del sistema sin tener que adivinar.

La resumimos mediante tres preguntas. La primera es si el sistema está vivo. Cada servicio informa si funciona y si puede comunicarse con su base.

La segunda pregunta es qué pasó. Para responderla utilizamos los registros, también llamados logs, que funcionan como la bitácora de cada servicio.

La tercera es por dónde pasó un pedido. Cada llamada lleva un número de seguimiento, como el número de guía de un paquete. No es el número del evento que vimos con Abdiel: aquel evita avisos duplicados y este nos permite seguir el recorrido.

Ese número permite saber por dónde pasó, a qué hora y en qué punto se detuvo. Sin él tendríamos que revisar cada servidor por separado.

Con observabilidad, buscamos un solo número y reconstruimos todo el recorrido.

Veamos cómo aplicamos estas ideas a un caso concreto, cuando un aviso no llega.

**SI SE TE OLVIDA, DI ESTO**

- Observabilidad es entender el sistema sin adivinar.
- Revisamos salud, registros y recorrido.
- Un número de seguimiento conecta toda la operación.

**SI PREGUNTAN**

**¿Esto es lo mismo que monitoreo?**

Son conceptos relacionados. El monitoreo avisa que algo anda mal y la observabilidad ayuda a entender la causa.

**SI HAY TIEMPO**

- Los registros del proyecto utilizan un formato estándar configurado fuera del código del negocio. Para activarlo solo necesitamos una línea de configuración.
- Cada servicio tiene una dirección especial que informa su salud. Kubernetes y la pantalla la consultan para mostrar si está disponible.

## 41. Un aviso no llegó: ¿cómo lo encontramos?

*Presenta: Moises*

**QUÉ DECIR**

Supongamos que alguien envió su solicitud, pero no recibió el aviso. No queremos revisar los registros de cada servicio uno por uno.

Por eso los reunimos en un solo lugar con tres herramientas. Promtail es el recolector: toma los registros de cada contenedor. Loki es la base de datos donde se guardan. Y Grafana es la pantalla donde los consultamos.

Escribimos el código de la solicitud y aparecen juntos los registros de ambos servicios, en orden: cuándo se envió, cuándo se entregó el evento y si notificaciones creó el aviso. Si el evento siguiera en la bandeja de salida, veríamos dónde se detuvo.

Un servicio encendido no siempre funciona bien. Por eso también revisamos resultados, como la cantidad de eventos pendientes.

Con este caso resuelto, resumamos lo que aprendimos sobre observabilidad.

**SI SE TE OLVIDA, DI ESTO**

- Promtail recolecta los registros.
- Loki es su base de datos y Grafana, la pantalla para consultarlos.
- Buscamos el código de la solicitud y aparecen los registros de los dos servicios.

**QUÉ HACER (NO SE DICE)**

- No hace falta explicar cómo se escribe la consulta en Grafana: basta la idea de buscar el código de la solicitud.

**SI PREGUNTAN**

**¿Grafana muestra métricas?**

Puede mostrarlas si le conectamos una fuente de métricas, como el consumo de memoria. En ARKA-Lite solo lo conectamos a Loki, así que lo usamos para consultar registros.

**SI HAY TIEMPO**

- El número de seguimiento se llama traceId y viaja en cada llamada entre los servicios. Además, cada evento tiene su propio identificador.
- Si todo aparece encendido y falta un aviso, revisamos la bandeja de salida y los registros del relevador y de notificaciones.
- Podemos buscar los registros de notificaciones que contengan la palabra evento. En segundos obtenemos las líneas relacionadas con sus números.

---

## 42. Observabilidad: lo que aprendimos

*Presenta: Moises*

**QUÉ DECIR**

De observabilidad nos llevamos tres ideas. Los registros deben organizarse desde el primer día, porque hacerlo más adelante cuesta mucho más.

En un sistema con varios servicios necesitamos un número que acompañe cada pedido de principio a fin. También debemos distinguir entre estar encendido y funcionar correctamente.

Por eso no revisamos únicamente la salud del proceso, sino también sus resultados. Si tuviéramos que comenzar por una sola práctica, usaríamos un número por operación en todos los registros.

Para cerrar, le paso la palabra a Abdiel, con los resultados.

**SI SE TE OLVIDA, DI ESTO**

- Organizamos los registros desde el inicio.
- Cada operación lleva un número de seguimiento.
- Estar encendido no siempre significa funcionar.

**SI HAY TIEMPO**

- Esta práctica también funciona con procedimientos. Podemos guardar el mismo número en los registros de la aplicación, del procedimiento y de cualquier otro sistema involucrado.

# Valor y cierre

## 43. Resultados para el negocio

*Presenta: Abdiel Barria*

**QUÉ DECIR**

Gracias, Moises. Lo que les contamos se traduce en estos resultados para el negocio.

Primero, los eventos pendientes permanecen guardados si se interrumpe la entrega, y reenviar el mismo evento no crea otro aviso.

Segundo, el trámite no se detiene. Aunque el servicio de avisos esté caído, la persona puede enviar su solicitud y el aviso llega cuando el servicio regresa.

Tercero, las reglas del negocio se cumplen y cada una tiene una prueba que la vigila.

Cuarto, Kubernetes puede reemplazar una copia que falla, mientras los eventos pendientes permanecen guardados en la base.

Por último, los cambios llegan con menos riesgo, porque detectamos los errores en minutos y antes de que afecten a los usuarios.

Estos resultados no dependen de la suerte, sino de varias decisiones que trabajan juntas.

**SI SE TE OLVIDA, DI ESTO**

- No se pierden eventos ni se duplican avisos.
- El trámite continúa aunque notificaciones esté fuera de servicio.
- Las pruebas y la recuperación reducen el riesgo.

**SI HAY TIEMPO**

- Comprobamos estos resultados en nuestro laboratorio y no en producción. Lo importante es que cada resultado se puede repetir y verificar.

---

## 44. Lo que sostiene los resultados

*Presenta: Abdiel Barria*

**QUÉ DECIR**

Detrás de cada resultado hay una práctica que aprendimos y aplicamos. La transacción, la bandeja de salida y el control de duplicados mantienen consistentes los datos.

La configuración está separada del código, así que podemos cambiar las claves sin modificar la aplicación.

También ejecutamos el mismo sistema en distintos ambientes mediante un solo comando. Las pruebas automáticas revisan cada cambio antes de incorporarlo.

Además, reunimos los registros en un solo lugar y relacionamos cada versión con el cambio que la produjo.

La mayoría de estas prácticas puede aplicarse sin reemplazar la tecnología que ya utilizamos.

Si tuviéramos que resumir todo el recorrido en tres ideas, serían las siguientes.

**SI SE TE OLVIDA, DI ESTO**

- Cada resultado se apoya en una práctica concreta.
- Datos, configuración, pruebas y registros trabajan juntos.
- Podemos aplicar estas ideas sin cambiar de tecnología.

**SI HAY TIEMPO**

- Un orden razonable sería comenzar con una tabla de pendientes en el proceso que más falla, agregar idempotencia donde existen reintentos y después incorporar un número de seguimiento.
- Ninguna de estas prácticas exige Java, microservicios ni Kubernetes. Son decisiones de diseño aplicables a los sistemas actuales.

---

## 45. Lo que nos llevamos

*Presenta: Abdiel Barria*

**QUÉ DECIR**

De toda la capacitación nos llevamos tres ideas. La primera es diseñar pensando en la falla, porque algún componente fallará y debemos definir cómo responderá el sistema.

La segunda es automatizar las revisiones y dejar la infraestructura descrita en archivos. Así podemos hacer cambios con mayor confianza y menor riesgo.

La tercera es hacer visible el sistema. No podemos diagnosticar ni explicar con claridad lo que no vemos.

Ese es el aprendizaje que conecta todo el recorrido: prepararnos para las fallas, revisar los cambios y poder entender qué ocurre en el sistema.

**SI SE TE OLVIDA, DI ESTO**

- Diseñamos pensando en las fallas.

- La automatización nos permite cambiar con confianza.
- Lo visible se puede diagnosticar y explicar.

**SI HAY TIEMPO**

- Si conservamos una sola idea, debe ser esta. Diseñemos qué hará el sistema cuando algo falle y no solamente cómo funcionará cuando todo salga bien.

---

## 46. Gracias

*Presenta: Abdiel Barria*

**QUÉ DECIR**

Comenzamos con una sola operación en Java y terminamos con un sistema completo de dos servicios, cada uno con su propia base de datos.

El sistema quedó empaquetado, instalado, vigilado y documentado. Además, puede conservar la información y recuperarse cuando una de sus partes falla.

Muchas gracias. Con gusto respondemos sus preguntas.

**SI SE TE OLVIDA, DI ESTO**

- Pasamos de una operación a un sistema completo.
- El sistema sabe conservar los datos y recuperarse.
- Muchas gracias. Escuchamos sus preguntas.

**QUÉ HACER (NO SE DICE)**

- Abdiel recibe las preguntas y se las pasa a quien presentó esa parte.